CS3230 : Design and Analysis of Algorithms (Spring 2014)
Tutorial Set #1
[SOLUTION SKETCHES]

Note (by LHW): Only for this first tutorial, I will be kind and explain in great details. For future tutorials, you will need to "grow up", as the explanation will be more like what we expect a CS3230 student to be able to understand (and fill in the gaps, if necessary).

***Routine Practice Problems** -- do not turn these in -- but make sure you know how to do them.*

R1-(a) Show, by definition, that $f(n) = O(n)$, where $f(n) = 2014n$.

ANSWER: We need to find c and n_0, and show that
$f(n) \le c\, n$ for all $n \ge n_0$. (**)
So, we first write down what we know
$f(n) = 2014n$
$\le 2014\, n$, for all $n \ge 1$, (here, we are lucky; we got it *for free*)
We choose $c = 2014$, $n_0 = 1$. Then (**) holds.
Hence, $f(n) = 2014n = O(n)$.

R1-(a') Is $f(n) = O(\lg n)$? **NO** Is $g(n) = O(n^2)$? **YES** Is $g(n) = O(n^{2014})$? **YES**

ANSWER: Suppose (by contradiction), that we assume that $f(n) = O(\lg n)$.
Now, what does that mean?
By defn, it means that we can find c and n_0, such that $f(n) \le c(\lg n)$, whenever $n \ge n_0$.
Then, it means $2014n \le c \lg n$, when $n \ge n_0$.
Or equivalently, $2014\left(\frac{n}{\lg n}\right) \le c$, even for very large n.
But, $\lim_{n\to\infty} 2014\left(\frac{n}{\lg n}\right) = \infty$.
Thus, when n is large, $2014\left(\frac{n}{\lg n}\right) > c$, for any constant c. This gives a contradiction.

R1-(c) Show, by definition, that $h(n) = O(n^2)$, where $h(n) = 25n^2 + 2014n$.

ANSWER: $h(n) = 25n^2 + 2014n$
$\le 25n^2 + 2014n^2$, for all $n \ge 1$ ($2014n \rightarrow 2014n^2$)
$= 2039n^2$, for all $n \ge 1$,
We choose $c = 2039$, $n_0 = 1$, we have $h(n) \le 2039n^2$, for all $n \ge 1$,
Hence, $h(n) = 25n^2 + 2014n = O(n^2)$.

(**Note:** You do *not* have to find the smallest c and the smallest n_0.
Any c and n_0 that work will be equally good. I find an easy one that works.
If you want a smaller value for c, you will need to work hard and may also have to use a bigger n_0. Remember, the burden of proof is on YOU to prove your result.)

R1-(c') Is $h(n) = O(n)$? **NO** Is $h(n) = O(n^3)$? **YES** Is $h(n) = O(n^{2039})$? **YES**

R1(b), (b'): DIY (Do It Yourself) Similar to 1(a), (a').
R1(d), (d'): DIY. Similar to (c), (c').

R2. Will only show R1-(c). DIY for the rest.

R2-(c): Show, by definition, that $h(n) = \Theta(n^2)$, where $h(n) = 25n^2 + 2014n$.

ANSWER:

Upper Bound:

For R1(c) above, we already know, $h(n) \le 2039\, n^2$, for all $n \ge n_0=1$,

Lower Bound:

Next, we need to do the "lower bound" proof. Which is also easy.

Since $h(n) = 25n^2 + 2014n \ge 25n^2$, for all $n \ge 1$,

Put these together, we have

$25n^2 \le h(n) \le 2039n^2$, for all $n \ge 1$,

We choose $C_1 = 25$, $C_2 = 2039$, $n_0 = 1$.

Hence, $h(n) = 25n^2 + 2014n = \Theta(n^2)$.

R2-(c') Is $h(n) = \Theta(n)$? **NO** Is $h(n) = \Theta(n^3)$? **NO** Is $g(n) = \Theta(n^{2039})$? **NO**

R3. From R1 and R2, we see that O is only an upper bound and so it is less precise, but Θ is a more precise (lower and upper) bound on the time complexity.

R4. [Reuse from HW1] Show that (a) and (b) are true:

(a) $\sum_{k=1}^{N}(\ln k) = \Theta(N \lg N)$ **(b)** $\sum_{k=1}^{N}(k) = \Theta(N^2)$

(c) Where, in HW1, have you seen algorithms with these running time analyses?

(a) Method 1: Using Stirling' Approximation [CLRS, p57]

$\sum_{k=1}^{N}(\ln k)$ $= (\ln 1) + (\ln 2) + (\ln 3) + \ldots + + (\ln N)$

$= (\ln (N!))$ (you *better* know this!)

$= N \ln (N) - N + O(\ln(N))$ (by the Stirling's approximation)

$= \Theta(N \ln (N)) = \Theta(N \lg (N))$ (**Note:** $\ln N = k (\lg N)$, for constant k)

Stirling's Approximation : $\ln(n!) = n\ln(n) - n + O(\ln(n))$

Stirling's Approximation: $n! \sim \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$

Method 2: By bounding summations.

Upper bound: $TT = \sum_{k=1}^{N} (\ln k) \le \sum_{k=1}^{N} (\ln N) = N \ln N$

Lower bound:
Idea: Throw about the first $N/2$ smaller terms k=1,2,…,$N/2$. Let $M = N/2$

Then, $TT = \sum_{k=1}^{N} (\ln k) \ge \sum_{k=M+1}^{N} (\ln k) \ge \sum_{k=M+1}^{Q} (\ln M) = (M \ln M)$

Note that $(M \ln M) \ge \frac{1}{4}(N \ln N)$, for all $N \ge 2^4$ (after a bit of simple math shown below)
$(M \ln M) = \frac{N}{2} \ln(\frac{N}{2}) = \frac{N}{2}(\ln N - \ln 2) = N \ln N(\frac{1}{2} - \frac{\ln 2}{\ln N}) \ge \frac{1}{4}(N \ln N)$ for $N \ge 2^4$

So, use $c_1 = ¼$ and $c_2 = 1$ and $n_0 = 16$, we have
$\frac{1}{4}(N \ln N) \le \sum_{k=1}^{N} (\ln k) \le N \ln N$
Thus, $\sum_{k=1}^{N} (\ln k) = \Theta(N \ln N) = \Theta(N \lg (N))$

This summation is seen in analysis of Heapsort; **[CLRS, pp 159-160]**

(**Note:** There is yet a third method, using Estimation of Integrals. See bottom line of this page.)

(b) $\sum_{k=1}^{N} (k) = (1 + 2 + 3 + \ldots + (N-1) + N) = N(N+1)/2 = \Theta(N^2)$
[**Remember:** Gauss could do this before he was 10 years old.]

Method 2: By bounding summations. DO IT YOURSELF. Similar to (a).

Method 3: Using Estimation of Integrals.

This summation is seen in analysis of BubbleSort, InsertionSort, etc.

Bottom Line:

I have given your 3 different methods of showing these two sums (a) and (b).

They appear almost everyone in Analysis of Algorithms.

So know it like your A, B, C.

D0: (Two Important Processes in CS) [SS by HW]

(a),(b) **[DIY -- Do it with a Table (Excel table, maybe?)]**
IMPORTANT that you do it yourself. You will learn from doing it.

(c) Trivial, will NOT do it here;

(d) Most of the time $h(n) = d(n)$, except in some "boundary" cases, they differ by 1?
Q: Where are those boundary cases?

(e) $h(n) = \lceil \lg(n+1) \rceil$ and $d(n) = \lceil \lg n \rceil$. (They are equal MOST of the time.)
(Note: $\lceil \lg(n+1) \rceil = 1 + \lfloor \lg n \rfloor$)

D1. Solution sketch to HW1-S1.
(a) [by NN (from Fall 2013)]
Strategy: binary search/ divide and conquer/ guess the middle of the range,
Pseudo-code: Give pseudo-code or description
Complexity: Explicitly state (lg n) or lg(1000) in this case, to get worst case of 10 guesses.
(For general n, worst case is $\lceil \lg(n+1) \rceil$. Refer to T1-D0)
Answer: Who eats more: Steven ≥ Howard
Concise: Please give concise writing and explanation.

Discussion: What if we say 1 to 1023? Or 1 to 1024? Or 1 to 1025?

(b) [by KS (from Fall 2013)]
(b)(i) Algorithm will terminate. (**Why:** number of people standing always decreases!)
Number of synchronized steps = $\lceil \lg n \rceil$. (All logarithms are base-2.) **[See T1-D0]**.
Final number is n. (the number of students)
The number of additions is (n–1).

(b)(ii) Everything is the same, except
number of synchronized steps varies, ranges from $\lceil \lg n \rceil$ to (n–1).

Note: There is a simple one-line answer for #-of-additions:
"every time a pair is formed, one student sits down."

Discussion: Compare to knock-out tournament, winner takes points of both players.
Beginning: n points altogether (1 per player), *End*: all n points with the winner.

D2. Solution sketch to HW1-S2. [by NN, expanded by LHW]
(a) Algorithm is simple modification of BFS/DFS
To detect cycles, check whether an unexplored edge leads to a visited node.
If so, terminate the search – a cycle found.
Pseudo-code or description
Must clearly explain why $O(n)$ achieved: Need to check no more than n edges.

Simplification: At start of algorithm, if ($m > n$), then graph has a cycle, terminate.
Theorem: A tree on n vertices has at most (n–1) edges. (from Discrete Math, CS1231)

Discussion: What if we want you to print out the cycle that you find?
What if graph has multiple edges (a, b). No change to algorithm or complexity;
What if cycle can contain duplicate nodes/edges?

HW1-S3(b): Stable Marriage Theorem

Proof by Contradiction:
Assume that S is a stable matching, and that the pair (m*, w+) does NOT belong to S.
Let (m*, w') be the pair in S containing the man m*,
and let (m', w+) be the pair in S containing the woman w+.
Then, man m* prefers w* to w', and woman w+ prefers m* to m'.
Thus, S is unstable! This is a contradiction.

D3. [Proving O, Ω, Θ by definition]

Let $f(n) = 23n^2 + 7n$ $\quad$ $g(n) = 23n^2 - 2014n$
$h(n) = 16n(\lg n) - 8n + 2014$ $\quad$ $k(n) = 2^n + 8n^2$

(a) Prove the following by using the definitions of O, Ω, Θ. Namely, find the respective constants c, c_1, c_2, and the positive integer n_0.

(i) $f(n) = O(n^2)$ (ii) $f(n) = O(n^4)$ (iii) $g(n) = \Omega(n^2)$ (iv) $g(n) = \Omega(n)$

(v) $h(n) = O(n \lg n)$ (vi) $h(n) = \Omega(n \lg n)$ (vii) $h(n) = \Theta(n \lg n)$ (viii) $k(n) = \Theta(2^n)$

[Answer Sketches]

(i) $f(n) = 23n^2 + 7n \le 23n^2 + 7n^2$ for all $n \ge 1$. (change $7n$ to $7n^2$), true for all $n \ge 1$.
$\le 30n^2$ for all $n \ge 1$.
Choose $c = 30$ and $n_0 = 1$. Therefore, by definition, $f(n) = O(n^2)$.

(iii) $g(n) = 23n^2 - 2014n$
$= n^2(23 - 2014/n)$
$\ge n^2 (22)$ when $n \ge 2014$. (when $n \ge 2014$, $2014/n \le 1$)
Choose $c = 22$ and $n_0 = 2014$. Therefore, by definition, $g(n) = \Omega(n^2)$.

(v) $h(n) = 16n(\lg n) - 8n + 2014$
$\le 16n(\lg n) + 2014$ (throw away $-8n$), true for all n.
$\le 16n(\lg n) + 2014n(\lg n)$, for all $n \ge 1$. (increase 2014), true for all $n \ge 1$.
$\le 2030\, n(\lg n)$ for all $n \ge 1$.
Choose $c = 2030$ and $n_0 = 1$. Therefore, by definition, $h(n) = O(n(\lg n))$.

(vi) $h(n) = 16n(\lg n) - 8n + 2014$
$\ge 16n(\lg n) - 8n$ (throw away +2014), true for all n.
$= n(\lg n)\,[16 - 8/(\lg(n))]$ (increase 2014), true for all $n \ge 1$.
$\ge 12\, n(\lg n)$ for all $n \ge 2$. (when $n \ge 4$, $8/(\lg(n)) \le 4$)
Choose $c = 12$ and $n_0 = 4$. Therefore, by definition, $h(n) = \Omega(n(\lg n))$.

The proofs are much easier using the theorems or the polynomial rule or even L'Hopotal Rule.

D4. [Analysis of Algorithm Ace.] Analyze the following algorithm (called Ace) and give a *good estimate* of its running time (in Θ notation).

```
Procedure Ace (N) :
    1. Set A[k]=1 for all k=2,3,...,N                    // Initialize A to all "primes"
    2. for (p := 2; p <= N; p++) do begin
    3.    if (A[p] = 1) then Print p;
    4.    for (j := 2p; j <= N; j := j + p) do A[j] :=0;  // mark j as "not prime"
    5. end
```

[**Note:** The running time is NOT $\Theta(n^2)$.]

[Solution from Acer: but has not been cleaned up completely.]

For every p, the inner for-loop has $(N/p - 1)$ iterations.
Thus the total number of iterations is

$$
\begin{aligned}
&(N/2 - 1) + (N/3 - 1) + (N/4 - 1) + \ldots + (N/N - 1) \\
&< N/2 + N/3 + N/4 + \ldots + N/N \\
&= N\,(1/2 + 1/3 + 1/4 + \ldots + 1/N) \\
&< N \cdot H(N) \qquad \text{(where H(k) is the kth Harmonic number)} \\
&= N\,(\ln N - \ln(1)) \\
&= N \ln N
\end{aligned}
$$

Thus the time complexity is $O(N \ln N) = O(N \lg N)$.

Discussion: What is this algorithm doing?
Can we make it a little faster, in practice? Asymptotically faster?